Program 1

Write a code in PL/SQL to develop a trigger that enforces referential integrity by preventing the deletion of a parent record if child records exist.

```
CREATE OR REPLACE TRIGGER Prevent_Parent_delete
BEFORE DELETE ON Parent
FOR EACH ROW
DECLARE
   V_Count NUMBER;
BEGIN
      SELECT COUNT(*) INTO V_Count
      FROM Child
      WHERE Parent_id = :OLD.Parent_id;

   IF V_Count > 0 THEN
   RAISE_APPLICATION_ERROR(-20001, 'Cannot delet Parent
         record: Child records exist.');

   END IF;
END;
/
```

Program 2

Write a code in PL/SQL to create a trigger that checks for duplicate values in a specific column and raises an exception if found.

```
CREATE OR REPLACE TRIGGER check_duplicate
  BEFORE INSERT OR UPDATE ON employees
  FOR EACH ROW
  DECLARE
      v_count NUMBER;
  BEGIN
      SELECT COUNT(*) INTO v_count
      FROM employees
      WHERE email = :New.email;
  IF v_count > 0 THEN
   RAISE APPLICATION_ERROR(-20002, 'Duplicate email not allowed.');
      END IF;
  END;
```

Program 3

Write a code in PL/SQL to create a trigger that restricts the insertion of new rows if the total of a column's values exceeds a certain threshold.

```
CREATE OR REPLACE TRIGGER limit_total_Sales
BEFORE INSERT ON Sales
FOR EACH ROW
Declare
    V_total NUMBER;
    V_limit CONSTANT NUMBER := 100000;
BEGIN
    SELECT SUM(amount) INTO V_total FROM Sales;
IF V_total + :New.amount > V_limit THEN
RAISE_APPLICATION_ERROR(-20003, 'Cannot insert:-
                        total Sales exceed limit.')
    END IF;
END;
```

Program 4

Write a code in PL/SQL to design a trigger that captures changes made to specific columns and logs them in an audit table.

```
CREATE TABLE emp_audit (
    emp_id NUMBER,
    old_salary NUMBER,
    new_salary NUMBER,
    Changed_on DATE
);

CREATE OR REPLACE TRIGGER log_salary_change
AFTER UPDATE OF SALARY ON employees
    FOR EACH ROW

BEGIN

    INSERT INTO emp_audit (emp_id, old_salary, new_salary,
                                                changed_on )
VALUES(:OLD.employee_id, :OLD.salary, :New.salary, SYSDATE);

END;
/
```

Program 5

Write a code in PL/SQL to implement a trigger that records user activity (inserts, updates, deletes) in an audit log for a given set of tables.

```
CREATE TABLE audit_log (
        username VARCHAR(30),
        action_type VARCHAR(10),
        table_name VARCHAR2(30),
        action_date DATE
);

CREATE OR REPLACE TRIGGER record_user_activity
AFTER INSERT OR UPDATE OR DELETE ON employees
BEGIN
INSERT INTO audit_log ( username, action type, table_name, action_date )
VALUES (USER,
        CASE
            WHEN INSERTING THEN 'INSERT'
            WHEN UPDATING THEN 'UPDATE'
            WHEN DELETING THEN 'DELETE'
        END,
        'EMPLOYEES',
        SYSDATE);
END;
/
```

Program 7

Write a code in PL/SQL to implement a trigger that automatically calculates and updates a running total column for a table whenever new rows are inserted.

```
CREATE OR REPLACE TRIGGER Calc_running_total
  BEFORE INSERT ON Sales
  FOR Each Row
  DECLARE
      v_total Number;
  Begin
      select NVL(SUM Caround), 0) Into v_total From Sales;
  :NEW.running_total := v_total + :New.around;

  END;
/
```

Program 8

Write a code in PL/SQL to create a trigger that validates the availability of items before allowing an order to be placed, considering stock levels and pending orders.

```
CREATE OR REPLACE TRIGGER check_item_stock
BEFORE INSERT ON orders
FOR EACH ROW
DECLARE
    v_stock NUMBER;
BEGIN
    SELECT stock_qty INTO v_stock
    FROM items
    WHERE item_id = :new.item_id;
IF
    v_stock < :new.order_qty THEN
    RAISE_APPLICATION_ERROR (-20008, 'Not enough
                stock to place the order');
    END IF;
    END;
    /
```

Evaluation Procedure	Marks awarded
PL/SQL Procedure(5)	5
Program/Execution (5)	5
Viva(5)	5
Total (15)	15
Faculty Signature	